LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK

PERTEMUAN KE [9]
[Namespace]



Dosen Pengampu:
Syahru Rahmayuda, S.Kom., M.Kom

Asisten Praktikum:
[Andreanus Eko Putra]
[Dani Ali]

Disusun Oleh:

Nama	: Dimas Luthfi Salim
NIM	: H1101251004
Kelas	: A
Kelompok	: -

PROGRAM STUDI SISTEM INFORMASI
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS TANJUNGPURA
2026

LEMBAR PENGESAHAN

No.	Aspek	Paraf	Tanggal
1.	Telah dikerjakan oleh mahasiswa		
2.	Telah diperiksa oleh asisten		
3.	Telah disetujui oleh dosen		

DAFTAR ISI

LEMBAR PENGESAHAN 2
DAFTAR ISI 3
BAB I 4
PENDAHULUAN 4
1.1 Tujuan Praktikum 4
1.2 Dasar Teori 4
BAB II 6
ALAT DAN BAHAN 6
2.1 Perangkat Lunak 6
2.2 Perangkat Keras 6
BAB III 7
LANGKAH KERJA 7
3.1 Persiapan Environment 7
3.2 Proses Pembuatan Kode 7
3.3 Pengujian 8
BAB IV 9
HASIL DAN PEMBAHASAN 9
4.1 Kode Program 9
4.2 Hasil Output 11
4.3 Analisis dan Pembahasan 11
BAB V 12
KESIMPULAN DAN SARAN 12
5.1 Kesimpulan 12
5.2 Saran 12
DAFTAR PUSTAKA 13
LAMPIRAN 14

BAB I

PENDAHULUAN

1.1 Tujuan Praktikum

1. Memahami konsep namespace dan menerapkannya untuk mengelola pengorganisasian kelas lintas folder pada PHP modern.
2. Membuat struktur direktori proyek yang mematuhi standar autoloading PSR-4.
3. Menggunakan Composer sebagai pengelola dependensi serta memanfaatkan fungsi autoload untuk memuat kelas secara otomatis tanpa manual require.
4. Menerapkan penanganan kesalahan (error handling) menggunakan blok try-catch dan antarmuka Throwable agar program dapat menangani kendala secara aman.

1.2 Dasar Teori

1. Namespace

   Namespace adalah cara dalam PHP untuk mengelompokkan kelas, antarmuka (interface), fungsi, dan konstanta yang saling berkaitan. Fitur ini berfungsi untuk menghindari konflik penamaan ketika aplikasi menggunakan beberapa komponen atau pustaka dari pengembang yang berbeda.

   Contoh penggunaan:

```
namespace App\Models;
class User {
        // ...
}
```

2. Composer dan PSR-4 Autoloading

   Composer adalah alat pengelola dependensi (dependency manager) untuk bahasa pemrograman PHP. Standar PSR-4 (Autoloading Standard) mendefinisikan spesifikasi untuk memetakan namespace secara otomatis ke direktori file fisik, sehingga kelas-kelas dapat dimuat (loaded) secara otomatis tanpa perlu memanggil fungsi require atau include satu persatu.

   Konfigurasi pada composer.json :

```
{
   "autoload": {
      "psr-4": {
         "App\\": "src/"
      }
   }
}
```

3. Error Handling dengan Try-Catch dan Throwable

   Dalam pengembangan aplikasi modern, mekanisme penanganan kesalahan sangat penting untuk mencegah aplikasi mengalami fatal error yang dapat merusak pengalaman pengguna. Blok try-

catch menggunakan Throwable (antarmuka induk bagi seluruh exception dan error di PHP) memungkinkan program menangkap berbagai jenis kendala secara aman dan menampilkan pesan kesalahan yang terstruktur.

Contoh penggunaan:

```
{
   "autoload": {
      "psr-4": {
         "App\\": "src/"
      }
   }
}
```

BAB II

ALAT DAN BAHAN

2.1 Perangkat Lunak

No.	Nama Perangkat Lunak	Fungsi
1.	Visual Studio Code	Text editor untuk menulis kode
2.	MAMP/Laragon	Web server dan database server
3.	Web Browser	Menjalankan skrip PHP
4.		
5.		

2.2 Perangkat Keras

No.	Spesifikasi	Keterangan
1.	Laptop	Prosesor minimal Core i3
2.	RAM	Minimal 4 GB
3.	Penyimpanan	Minimal 50 GB ruang kosong
4.		
5.		

BAB III
LANGKAH KERJA

3.1 Persiapan Environment

Pertemuan 9 – Namespace:

1. Menyalakan Laragon dan memastikan berjalan.
2. Membuat folder baru di PPBO dengan nama Laprak9.
3. Membuka folder tersebut menggunakan Visual Studio Code.

3.2 Proses Pembuatan Kode

Pertemuan 9 – Namespace:

Soal 1 Analisis Kode

1. Mengapa namespace harus ditulis paling atas setelah <?php ?
   Berdasarkan aturan sintaks PHP, deklarasi namespace harus selalu diletakkan pada baris pertama di bawah tag pembuka <?php sebelum adanya instruksi kode lain, output, atau spasi kosong (whitespace). Hal ini bertujuan agar compiler PHP mengetahui di bawah ruang nama mana kelas, fungsi, atau konstanta tersebut didefinisikan sebelum skrip mulai memuat atau mengeksekusi kode lain.
2. Apa perbedaan antara Praktikum 1 (manual require) dengan Praktikum 2 (Composer autoload)?
   Di praktikum 1 (manual require), pengembang harus menuliskan pemanggilan berkas secara manual satu per satu menggunakan fungsi require_once untuk setiap kelas yang ingin digunakan. Jika struktur direktori berubah atau jumlah kelas bertambah banyak, pengembang harus memperbarui pemanggilan tersebut secara manual.
   Sedangkan di praktikum 2 (composer autoload), pemuatan berkas dilakukan secara otomatis oleh sistem autoloading PSR-4 milik Composer melalui satu berkas utama vendor/autoload.php. Pengembang tidak perlu lagi memanggil berkas kelas satu per satu secara manual karena Composer yang mengatur pemetaannya berdasarkan konfigurasi pada composer.json.
3. Kapan folder vendor/ muncul pada Praktikum 2?
   Folder vendor/ beserta seluruh isi berkas konfigurasinya muncul secara otomatis setelah perintah pembaruan autoload dijalankan di terminal, yaitu ketika mengeksekusi perintah composer dump-autoload atau composer install. Sebelum perintah tersebut dijalankan, folder vendor/ tidak akan ada di dalam direktori proyek.

Soal 2 Eksplorasi Mandiri:

Standar PHP Framework Interoperability Group (FIG) memainkan peran penting dalam menjaga keseragaman dan interoperabilitas kode dalam ekosistem PHP modern. PSR-0 merupakan standar autoloading generasi awal yang mewajibkan struktur direktori mereplikasi nama namespace secara penuh serta memperlakukan garis bawah (_) sebagai pemisah folder, namun standar ini kini telah usang (deprecated). Sebagai penyempurnaan, PSR-4 hadir sebagai standar autoloading modern yang lebih bersih dan efisien—seperti yang diterapkan pada Composer—di mana sebuah direktori dasar (misalnya src/) dapat dipetakan langsung ke dalam suatu namespace utama tanpa memerlukan struktur folder yang terlalu panjang. Di sisi lain, PSR-12 tidak berkaitan dengan autoloading, melainkan berfungsi sebagai panduan penulisan gaya kode (coding style guide) untuk memastikan konsistensi penulisan sintaks, aturan indentasi, penempatan kurung kurawal, serta penggunaan fitur-fitur PHP modern agar kode mudah dibaca dan dipelihara secara kolektif.

3.3 Pengujian

Pertemuan 9 – Namespace:

1. Menjalankan skrip tugas1.php melalui browser dengan URL http://php-namespace.test .
2. Memastikan output sesuai dengan yang diharapkan.
3. Mencatat hasil output.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Kode Program

Soal 2 Modifikasi Kode

Product.php

```php
<?php

namespace App\Models;

class Product {
    private string $name;
    private float $price;

    public function __construct(string $name, float $price) {
        $this->name = $name;
        $this->price = $price;
    }
    public function getName(): string {
        return $this->name;
    }
    public function getPrice(): float {
        return $this->price;
    }
    public function getFormattedPrice(): string {
        return 'Rp ' . number_format($this->price, 2, ',', '.');
    }
}
```

ProductService.php

```php
<?php

namespace App\Services;

use App\Models\Product;

class ProductService {
    public function createProduct(string $name, float $price): Product {
        return new Product($name, $price);
    }

    public function displayProduct(Product $product): string {
        return "<br>Nama Produk: " . $product->getName() . "<br>" .
               "Harga Produk: " . $product->getFormattedPrice();
    }
}
```

index.php

```php
<?php

require_once __DIR__ . '/vendor/autoload.php';

use App\Services\UserService;
use App\Services\ProductService;

try {
$pservice = new ProductService();
$uservice = new UserService();

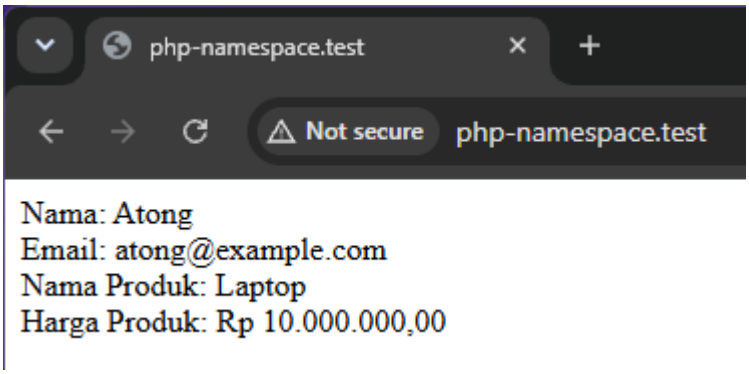
$user = $uservice->createUser(
    'Atong',
    'atong@example.com'
);

$product = $pservice->createProduct(
    'Laptop',
    10000000
);
echo $uservice->displayUser($user);
echo $pservice->displayProduct($product);

} catch (Throwable $e) {
    echo "<h3>Terjadi Error</h3>";
    echo "<p>Pesan: " . $e->getMessage() . "</p>";
    echo "<p>File: " . $e->getFile() . "</p>";
    echo "<p>Line: " . $e->getLine() . "</p>";
}
```

4.2 Hasil Output

Soal 2 Modifikasi Kode



4.3 Analisis dan Pembahasan

Paragraf Analisis:

Pada praktikum kali ini, penerapan konsep Namespace, Composer PSR-4 Autoloading, dan Error Handling berhasil diimplementasikan untuk membangun arsitektur aplikasi berbasis PHP modern yang terstruktur dan aman. Penggunaan namespace (App\Models dan App\Services) terbukti efektif dalam mengelompokkan kelas-kelas fungsional secara modular serta mencegah terjadinya konflik penamaan antar komponen. Melalui konfigurasi autoloading standar PSR-4 pada berkas composer.json, pemuatan kelas antar direktori (src/ dan vendor/) dapat berjalan secara otomatis tanpa memerlukan pemanggilan fungsi require_once secara manual satu persatu. Selain itu, penerapan mekanisme penanganan kesalahan menggunakan blok try-catch dengan antarmuka Throwable berhasil mencegah aplikasi mengalami fatal error ketika terjadi kendala pada instansiasi objek maupun pemanggilan metode, sehingga program mampu menangkap dan menampilkan informasi galat secara lebih informatif dan terstruktur. Hasil pengujian menunjukkan bahwa kombinasi namespace, autoloading Composer, dan error handling ini mampu menciptakan kode yang bersih, mudah dipelihara (maintainable), serta mencerminkan standar pengembangan aplikasi profesional saat ini.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Tuliskan kesimpulan dari praktikum yang telah dilakukan. Minimal 3 poin.

1. Konsep Namespace berhasil diimplementasikan untuk mengorganisasikan kelas-kelas ke dalam ruang nama yang terstruktur (seperti App\Models dan App\Services), sehingga dapat mencegah konflik penamaan pada aplikasi PHP modern.
2. Penggunaan Composer dengan standar autoloading PSR-4 terbukti efektif dalam memetakan direktori fisik (seperti src/) secara otomatis, sehingga pengembang tidak perlu lagi memanggil berkas kelas satu persatu secara manual menggunakan require_once.
3. Mekanisme penanganan kesalahan (error handling) menggunakan blok try-catch dengan antarmuka Throwable berhasil diterapkan untuk menangkap galat secara aman dan mencegah terjadinya fatal error yang dapat menghentikan eksekusi program secara paksa.
4. Kombinasi namespace, autoloading Composer, dan error handling berhasil menciptakan struktur proyek yang bersih (clean code), modular, serta mempermudah pemeliharaan sistem dalam pengembangan aplikasi berbasis objek profesional.

5.2 Saran

Tuliskan saran untuk perbaikan praktikum ke depannya.

1. Perlu ditambahkan validasi data yang lebih ketat pada metode constructor atau setter (seperti pengecekan format surel yang valid atau harga produk yang tidak boleh bernilai negatif) untuk menghindari anomali data.
2. Sebaiknya memanfaatkan fitur optimasi autoloading bawaan Composer saat tahap produksi (seperti menjalankan perintah composer dump-autoload -o) guna meningkatkan performa pemuatan kelas pada aplikasi berskala besar.
3. Perlu memperluas eksplorasi penanganan kesalahan dengan membuat Custom Exception tersendiri di luar Throwable bawaan untuk menangani kasus bisnis yang lebih spesifik pada modul layanan.

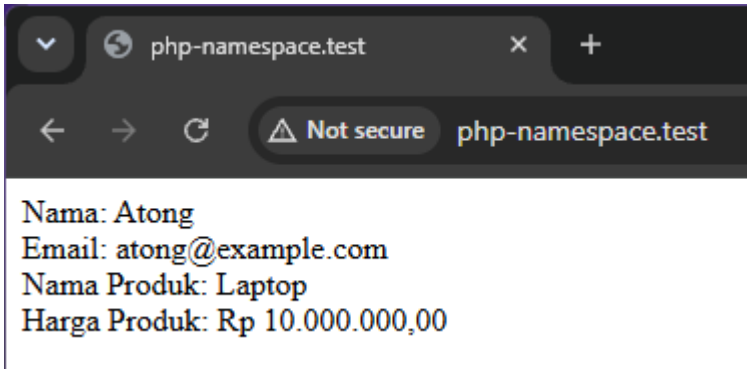
DAFTAR PUSTAKA

1. Welling, L., & Thomson, L. (2017). *PHP and MySQL Web Development* (5th ed.). Pearson.
2. Butler, T., & Yank, K. (2017). *PHP & MySQL: Novice to Ninja* (6th ed.). SitePoint.
3. Nixon, R. (2018). *Learning PHP, MySQL & JavaScript: With jQuery, CSS & HTML5* (5th ed.). O'Reilly Media.
4. PHP Documentation Group. (2026). *Classes and Objects*. https://www.php.net/manual/en/language.oop5.basic.php

LAMPIRAN

Lampiran 1. Screenshot Hasil Praktikum

Soal 2 Modifikasi Kode



Lampiran 2. Source Code Lengkap

Soal 2:

index.php

php-namespace > index.php

```php
<?php

require_once __DIR__ . '/vendor/autoload.php';

use App\Services\UserService;
use App\Services\ProductService;

try {
$pservice = new ProductService();
$uservice = new UserService();

$user = $uservice->createUser(
    'Atong',
    'atong@example.com'
);

$product = $pservice->createProduct(
    'Laptop',
    10000000
);
echo $uservice->displayUser($user);
echo $pservice->displayProduct($product);

} catch (Throwable $e) {
    echo "<h3>Terjadi Error</h3>";
    echo "<p>Pesan: " . $e->getMessage() . "</p>";
    echo "<p>File: " . $e->getFile() . "</p>";
    echo "<p>Line: " . $e->getLine() . "</p>";
}
```

Product.php

php-namespace > src > Models > Product.php

```php
<?php

namespace App\Models;

class Product {
    private string $name;
    private float $price;

    public function __construct(string $name, float $price) {
        $this->name = $name;
        $this->price = $price;
    }
    public function getName(): string {
        return $this->name;
    }
    public function getPrice(): float {
        return $this->price;
    }
    public function getFormattedPrice(): string {
        return 'Rp ' . number_format($this->price, 2, ',', '.');
    }
}
```

ProductService.php

php-namespace > src > Services > ProductService.php

```php
<?php

namespace App\Services;

use App\Models\Product;

class ProductService {
    public function createProduct(string $name, float $price): Product {
        return new Product($name, $price);
    }

    public function displayProduct(Product $product): string {
        return "<br>Nama Produk: " . $product->getName() . "<br>" .
               "Harga Produk: " . $product->getFormattedPrice();
    }
}
```